

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	P. Sai Charan Tej	Reg. No.:	192465048
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Analyze the scalability of Merge Sort by applying it to datasets of increasing sizes (e.g., 10, 107, 10? elements). Record execution time and plot the growth trend. Interpret the results using asymptotic analysis and explain how the divide and conquer approach ensures predictable performance. Justify its suitability for large-scale data processing

Code of Conduct

I P.Sai Charan Tej(192465048)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Language: **Python** C C++ Java

QUESTIONS

Q1

Merge Sort
100 marks

1/1 answered

Q1 Merge Sort

100 marks

Analyze the scalability of Merge Sort by applying it to datasets of increasing sizes (e.g., 1000, 10000, 100000 elements). Record execution time and plot the growth trend. Interpret the results using asymptotic analysis and explain how the divide and conquer approach ensures predictable performance. Justify its suitability for large-scale data processing.

Your Code

```

1 import random
2 import time
3 def merge_sort(arr):
4     if len(arr) > 1:
5         mid = len(arr) // 2
6         left = arr[:mid]
7         right = arr[mid:]
8
9         merge_sort(left)
10        merge_sort(right)
11
12        i = j = k = 0
13
14        while i < len(left) and j < len(right):
15            if left[i] < right[j]:
16                arr[k] = left[i]
17                i += 1
18            else:
19                arr[k] = right[j]
20                j += 1
21            k += 1
22
23        while i < len(left):
24            arr[k] = left[i]
25            i += 1
26            k += 1
27
28        while j < len(right):
29            arr[k] = right[j]
30            j += 1
31            k += 1
32
33    sizes = [1000, 10000, 100000]
34
35    print("Dataset Size\tExecution Time (seconds)")
36
37    for size in sizes:
38        data = random.sample(range(size + 10), size)
39
40        start = time.perf_counter()
41        merge_sort(data)
42        end = time.perf_counter()
43
44        print(size, "\t\t", round(end - start, 3))

```

Submitted

Input

stdin input

Output

Dataset Size	Execution Time (seconds)
1000	0.003139
10000	0.041828
100000	0.28884